PYTHON FULL STACK

Python full-stack developer is a **web developer** who uses **Python** as their main **programming language** to handle all aspects of building a **web application**. They build the **user interface** that is the **front end** as well as the **back end** of the website where **server logic** is stored using **Python**. For **front-end development**, they generally use languages like **HTML**, **CSS**, and **JavaScript**.

For the **backend** tasks frameworks like **Django** or **Flask** are used. These developers bridge the gap between the **client** and the **server** which ensures the website is fully functional across the entire application.

Key Responsibilities

- **Front-End Development:**
  - Building user interfaces (UI) using **HTML, CSS, and JavaScript frameworks.**
  - Ensuring **responsiveness** and **functionality** across different devices.
- **Back-End Development:**
  - Writing **Python code** to handle **server-side logic** and **data processing**.
  - Interacting with **databases** to **store**, **retrieve**, and **manipulate data**.
  - Implementing **APIs (Application Programming Interfaces)** for **communication** with other **applications**.
- **Full-Cycle Development:**
  - Takes part in the entire **development lifecycle**, from planning and design to implementation, testing, and deployment**.**

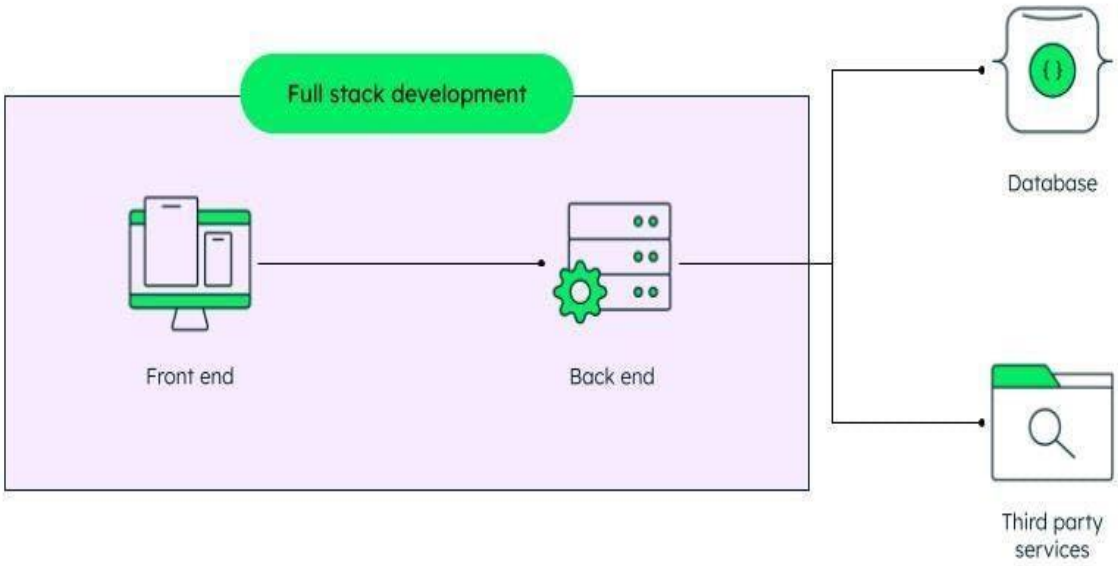
Other than these there are some **Python-Specific Tasks** such as

- Writing **clean**, **efficient**, and **maintainable Python code**.

Using **Python libraries** and **frameworks** for **back-end development** (e.g., **Django**, **Flask**).

-

TO



BECOME A PYTHON FULLSTACK DEVELOPER

1. Master Python Fundamentals

Before building websites from start to finish, you've got to understand the basics of Python. The foundation that helps you handle all the different parts of web development smoothly. You will have to learn about various Python frameworks and libraries and hence a solid foundation will make the learning journey easier.

- **Essential Building Blocks:** Grasp core concepts like **data types** (text, numbers, etc.), **variables** (data storage containers), **loops** (repetitive code execution), and functions (reusable blocks of code).
- **Problem-Solving Skills:** Practice applying these fundamentals to solve **coding challenges** and build small programs.
- **Preparation for Advanced Topics:** A solid base in **Python** paves the way for learning more complex **full-stack development** concepts.

2. Learn Front-End Technologies

-

The front end is part of the website which user interact with that is also known as the **user interface**. For the creation of a full-stack website that can handle **clientside logic** as well you can explore **front-end frameworks of Python**. The frameworks make it very easy to create the back-end architectures.

- **Core technologies:** Master **HTML** (structure and content organization), **CSS** (styling like colors, fonts, and layouts), and **JavaScript** (interactivity like animations and user input handling).
- **Enhance interactivity:** Learn to handle **user input**, **animations**, and **complex layouts** to create a **user experience** that is not only visually appealing but also engaging and responsive.

3. Explore Back-End Frameworks

The **back end** is part of the website where **server logic** is stored and the **data** from the web application is processed. For the creation of a **full-stack website** that can handle **server-side logic** as well, you can explore **back-end frameworks of Python**. The frameworks make it very easy to create the back-end architectures.

- **Python Frameworks**: Popular frameworks like **Django** and **Flask** are pre-made structures for **back-end development**. They offer a structured approach and various tools to build robust web applications efficiently.
- **Server-Side Logic**: **Routing** directs users to different sections (pages) of your application based on the URL. **Models** represent the real-world data your application uses (e.g., users, products). **Views** are responsible for presenting this data, while **templates** allow you to combine dynamic data with HTML for flexible content creation.
- **Scalability and Security**: These frameworks are designed to scale efficiently and handle high user traffic volumes. Additionally, you'll learn best practices for securing your application to protect user data and prevent hacking attempts.

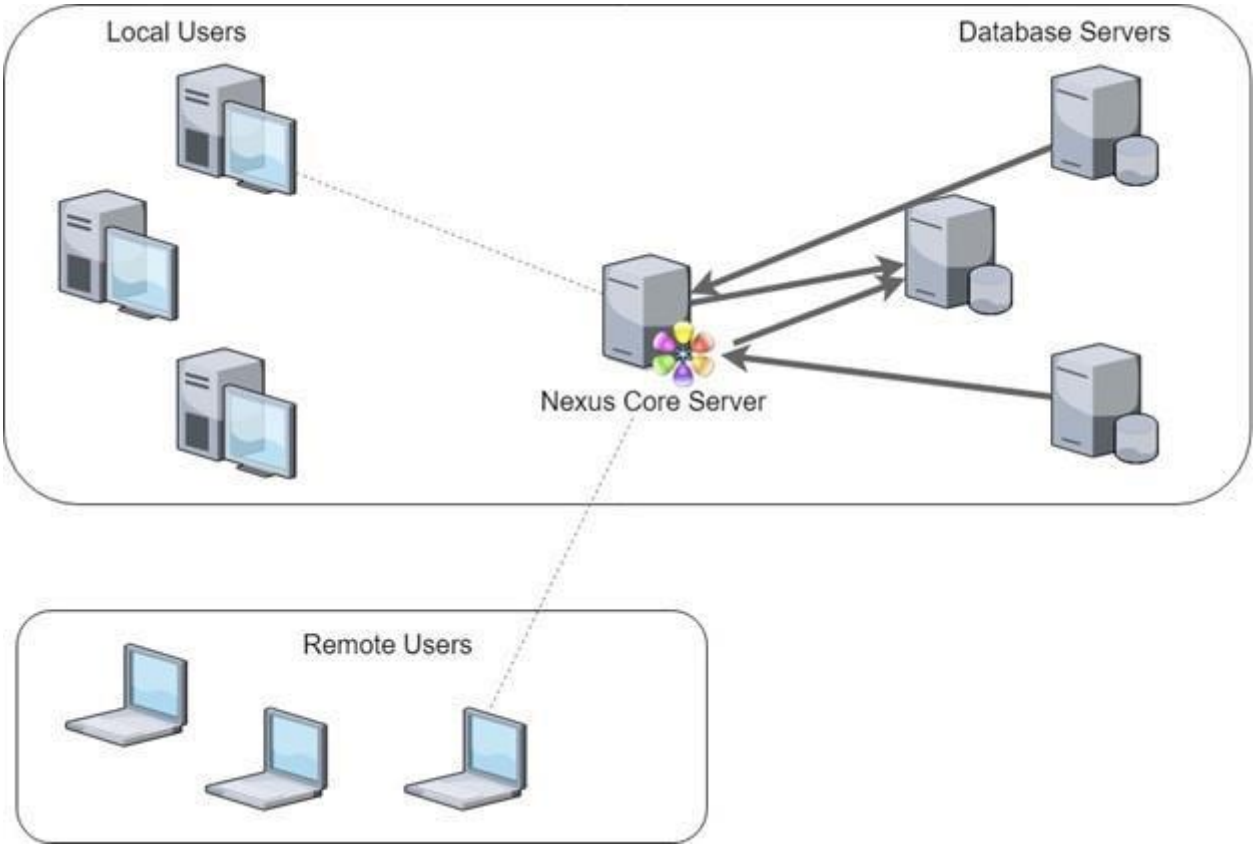
4. Understand Database Management

-

Knowing how to work with **databases** is essential for building the backend of your web applications. Databases play a fundamental role in web applications. They serve as repositories for storing and managing data efficiently. The **database management** allows you to design, implement, and maintain databases that support your application**.**

- **Database Systems**: Databases store all the information your application needs (e.g., user accounts, product details, purchase history) in a structured format. Popular options like **PostgreSQL**, **MySQL**, and **MongoDB** offer different strengths and functionalities.
- **SQL Power**: **SQL (Structured Query Language)** is a special language for talking to your database. Using this, you can write commands to add, edit, and retrieve data as needed, ensuring that the application has access to the stored information.
- **Object-Relational Mapping (ORM)**: These Python libraries simplify data interaction by working with objects that represent your data structure. They eliminate the need to write raw SQL queries for most tasks, making development faster and easier to maintain.

Client Server Architecture



-

The Client-server model is a distributed application structure that partitions tasks or workloads between the providers of a resource or service, called servers, and service requesters called clients. In the client-server architecture, when the client computer sends a request for data to the server through the internet, the server accepts the requested process and delivers the data packets requested back to the client. Clients do not share any of their resources. Examples of the Client-Server Model are Email, World Wide Web, etc.

FRONT END DEVELOPMENT

Front-end is the part that is accessible to the user only, It should be selfexplanatory and must be user-friendly and designed. so to achieve this we have some basic languages which can be used to create interactive web pages. These are the basic needs for the creation of a web page.

- **HTML:** It was introduced in 1993 and currently at version HTML5, is a foundational language for creating structured web content. It facilitates the

creation of forms, tables, and structured data, essential for input elements and organizing information on web pages. HTML's evolution continues to shape the internet by enabling developers to design interactive and accessible user interfaces across various platforms and devices.

- **CSS:** CSS, introduced on 17 December 1996 and currently at version CSS3, stands for Cascading Style Sheets. It complements HTML by styling web pages, managing colors, and enabling the creation of responsive designs. CSS empowers web developers to enhance user interfaces, ensuring visually appealing and consistent layouts across different devices and screen sizes.
- **JavaScript**: ECMAScript, introduced on 4 December 1995, is a lightweight, crossplatform, single-threaded programming language. It enables dynamic changes and event handling in web
-

development, making it essential for creating interactive and responsive user interfaces.

Responsive Web Development

Specifies that web applications should be accessible on devices of various shapes and sizes, particularly crucial for mobile devices due to increasing traffic.

Python Table of Content:

- File handling
- OOP
- Exception Handling

DJango Table of Content

- Overview
- Basics
- Environment
- Create project
- Create app
- App life cycle
- Create views
- URL mapping
- Templates
- Models
- Page Redirection
- Form processing

DATABASE

Introduction to database

- DBMS vs RDBMS
- Properties of RDBMS
- Introduction to SQL
- SQL sub languages
- DDL
- DML
- DRL/QL
- DCL
- TCL

•

HTML

- Introduction
- Features
- Structure
- HTML Tags
- Forms

CSS

- Introduction
- Need for CSS
- Types of CSS
- Selectors

JAVA SCRIPT

- Introduction
- Data types Dialog boxes
- Input and Output statements
- Functions
- Arrays
- Strings
- DOM
- Class
- Object
- JSON

1.HTML

- HTML stands for Hyper Text Markup Language
- It works with HTTP protocol

•

- it used to create web pages.
- It contains tags and hyperlinks.

1.1History of HTML

- The first version of HTML was written by **Tim Berners-Lee** 1991
- The most widely used version throughout the 2000's was HTML 4.01, which became an official standard in December 1999.

1.2HTML Element

An HTML element is defined by a start tag, some content, and an end tag.HTML Tag Reference.

Tag	Description
<html>	Defines the root of an HTML document
<body>	Defines the document's body
<h1> to <h6>	Defines HTML headings

1.3Target Attributes

The target attribute **specifies a name or a keyword that indicates where to display the response that is received after submitting the form**.

- Attribute provides additional information about elements.
- Attributes are always specified in the start tag.
- Attributes usually come in name/value pairs like: **name="value" Example:**

```
<a href="https://www.w3schools.com">Visit W3Schools</a>
```

In this above example href is the Attribute. It specifies the address of the destination

Example:

```
<p style = "colour:red;" >This is a red paragraph. </p >
```

-

In the above example style is an attribute used to apply styles to the HTML elements.

1.4HTML Frames

HTML frames are used to divide your browser window into multiple sections where as each section can load a separate HTML document

The <frame>tag was used in HTML 4 to define one particular window (frame) within a <frameset>.

1.5HTML Forms

An HTML form is used to collect user input. The user input is most often sent to a server for processing**.**

The HTML <form> Elements

The HTML <form>element can contain one or more of the following form elements:

- <input>
- <label>
- <select>
- <textarea>
- <button>
- <option>

The <input> Element

- The HTML <input>element is the most used form element.

- An <input>element can be displayed in many ways, depending on Here are some examples:

•

Example:

```
<form>
<label for="fname">First name:</label><br>
<input type="text" id="fname" name="fname"><br>
<label for="lname">Last name:</label><br>
<input type="text" id="lname"  name="lname"> </form>
```

2.CSS

- CSS stands for Cascading Style Sheet
- CSS used to style an HTML document
- CSS describes how HTML elements are displayed on screen

2.1Types of CSS

1. **Inline** - by using the style attribute inside HTML elements
2. **Internal** - by using a <style> element in the <head> section
3. **External** - by using a <link> element to link to an external CSS file

Inline CSS

- An inline CSS is used to apply a unique style to a single HTML element.
- An inline CSS uses the style attribute of an HTML element. **Example**

```
<h1 style="color:blue;font-weight:bold;">A Blue Heading</h1>
<p style="color:red;">A red paragraph.</p>
```

Internal CSS

- An internal CSS is used to define a style for a single HTML page.
- It is defined in the <head> section of an HTML page, within <style> element.

Example

•

```
<html>
    <head>
        <style> body
        {background-color:
            powderblue;} h1 {color:
            blue;}
            p{color: red;} </style>
    </head>
        <body>
      <h1>This is a heading</h1>
     <p>This is a paragraph.</p>
        </body>
        </html>
```

External CSS

- An external style sheet is used to define the style for many HTML pages.
- To use an external style sheet, add a link to it in the <head> section of each HTML page: **Example**

```
<html>
  <head>
<link rel="stylesheet" href="styles.css">
</head>
<body>
 <h1>This is a heading</h1>
 <p>This is a paragraph.</p>
</body>
</html>
```

The external style sheet can be written in any text editor. The file must not contain any HTML code, and must be saved with a .css extension. Here is what the "styles.css" file looks like:

"styles.css":

-

```
body {background-
color:powderblue; }  h1{
color:blue;  } p{color:red;}
```

2.2Types of Selectors

- Element Selector
- ID selector
- Class Selector
- Grouping Selector **Element Selector:**

□ •Element selectors selects the elements based on element names.

Example

```
h1 {
text-align:
left; color:
  green;
 }
```

○ ID Selector:

•ID selector uses the ID attribute of an HTML element to select a specific element.

•ID should be unique within the webpage

•Need to prefix a hash(#) to the id of the attribute.

Example

```
<h1 id="Header1">Hello World!!!</h1>.
      Then the selector should be
      #Header1    {
            font-
            size:
            20px;
            color:
            green
                  }
```

•

Class Selector:

- The class selector selects elements within a specific class attribute
- Class name should be prefixed with period(.)

Example suppose the following is the HTML content

 This is class selector

Then the selector should be

.

```
Paragraph {
color:
Green;
textalign:
center;
background-color: red;
}
```

Grouping of Selectors:

•Multiple selectors can be grouped into a single style declaration •Selector1,selector2,… { Property1: value;

property2: value;

…..}

Ex: h1,h2,p{ color: green; font-size:20px}

Universal selector

•Universal selectors are used to select any element (styles Applied to all elements in the web page)

•The selector name should "*"

•EX:- If the HTML content is as given below

<h1>html</h1>

hyper text markup language

and the

selector is

•

* {**color: blue; textalign: center; background-color: yellow; }**

3.JAVASCRIPT

3.1History of JavaScript

- JavaScript was **invented by Brendan Eich in 1995**
- It was developed for Netscape2, and became the ECMA-262 standard in 1997

In September 1995, a Netscape programmer named Brendan Eich developed a New scripting language in just 10 days.

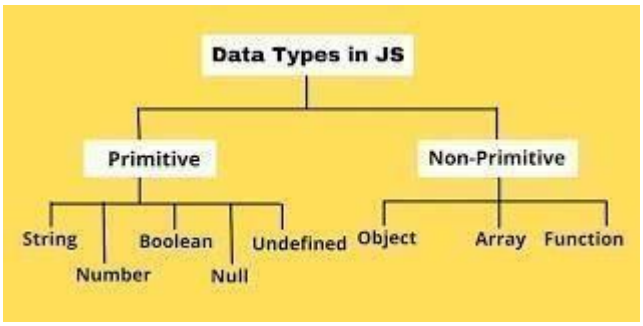
3.2variable declaration and datatypes in java script

Variable declaration:

Ex: var a; or let a;

Datatypes:

JavaScript is loosely typed. Because variable doesn't restricted to the datatypes.



3.3Operators in JavaScript

JavaScript operators are symbols that are used to perform operations.

There are following types of operators in JavaScript:

1. Arithmetic operators
2. Relational operators
3. Bitwise operators
4. Logical operators

•

5. Assignment operators
6. Special operators

3.4Conditional Statements, loops, and String handling functions:

Conditional statements:

Conditional statements are used to perform different actions based on different conditions.

In java script we have the following conditional statements:

1. **If:** if is used to specify a block of code to be executed, if a specified condition is true **Syntax:**

if (condition) {
Block of code to be executed is the condition is true
}

2. **Else if:** else if is used to specify a new condition to test, if the condition is false.

Syntax:

if (condition1) { }
else if (condition2) {
***// block of code to be executed if the condition1 is false and condition2 is true* } else {**
***// block of code to be executed if the condition1 is false and condition2 is false* }**

3. **Nested else if: you can have if statements inside if statements, this is called a nested if.**

Syntax: if condition1 {
// code to be executed if condition1 is true if *condition2*
{
// code to be executed if both condition1 and condition2 are true }

•

else {}

4. **Switch:** Use the switch statement to select one of many code blocks to be executed.

Syntax:

```
Switch(expr
e ssion) {
casex:
//codeblock
b reak;
 case y:
  // code block
break; default:
 //code block
}
```

Loops: Loops can execute a block of code number of times

1. **For:** loops through a block of code a number of times **Syntax:**

```
for (expression 1; expression 2; expression 3)
 {
   // code block to be executed//
 }
```

For/in: loops through the properties of an object

Syntax:

```
for (key in
 object)
  {
 // code block to be executed
  }
```

For/of: for/of statement loops through the values of an iterable object.

Syntax:

•

```
for (variable of iterable)
{
// code block to be executed
}
```

2. **While:** The while loop loops through a block of code as long as a specified condition is true **Syntax:**

```
while (condition)
{
  // code block to be
executed }
```

3. **Do while:** The do while loop is a variant of the while loop. This loop will execute the code block once, before checking if the condition is true, and then it will repeat the loop as long as the condition is true.

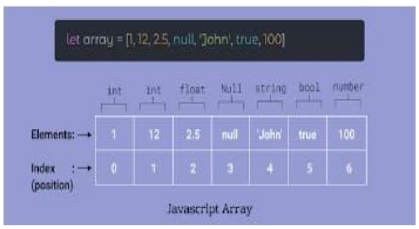
Syntax: do {

```
// code block to be
executed } while (condition
```

String length	String trim()
String slice()	String trimStart()
String substring()	String trimEnd()
String substr()	String padStart()
String replace()	[illegible]
String replaceAll()	String charAt()
String toUpperCase()	String charCodeAt()
String toLowerCase()	String split()
String concat()	

3.5 Arrays

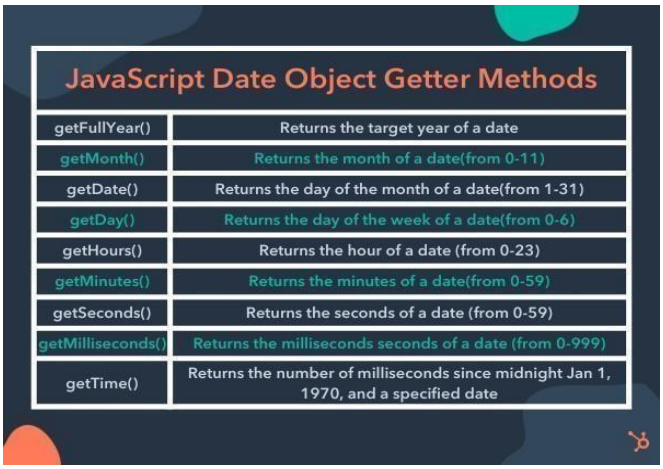
An array is a special variable, which can hold more than one variable.



3.6 JavaScript Date methods:

The **JavaScript date** object can be used to get year, month and day. You can display a timer on the webpage by the help of JavaScript date object. You can use different Date constructors to create date object. It provides methods to get and set day, month, year, hour, minute and seconds.

•



3.7JavaScript regular expressions:

- A regular expression is a sequence of characters that forms a **search pattern**.
- Regular expressions are patterns used to match character combinations in strings.In JavaScript, regular expressions are also objects.

```
var str = "EduCBA";
var regEx = /^[A-Za-z]/;
var res = "false";
if(str.match(regEx)){
res= "true";
}
alert(res);
```

www

- Regular expressions can be used to perform all types of **text search** and **text replace** operations.
- A regular expression can be a single 3character, or a more complicated pattern.

3.8HTML events and DOM:

Html events:

- HTML events are **"things"** that happen to HTML elements.

- When JavaScript is used in HTML pages, JavaScript can **"react"** on these events.

•

An HTML event can be something the browser does, or something a user does.

Here are some examples of HTML events:

1. An HTML web page has finished loading
2. An HTML input field was change
3. An HTML button was clicked

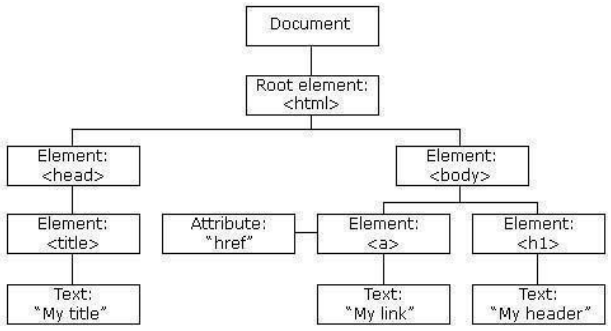
DOM (document object model):

When a web page is loaded, the browser creates a **D**ocument **O**bject **M**odel of the page.

HTML DOM methods are **actions** you can perform (on HTML Elements).

HTML DOM properties are **values** (of HTML Elements) that you can set or change.

The **HTML DOM** model is constructed as a tree of **Objects:**



•

PYTHON:

What is Python

Python is a general-purpose, dynamically typed, high-level, compiled and interpreted, garbage-collected, and purely object-oriented programming language that supports procedural, object-oriented, and functional programming.

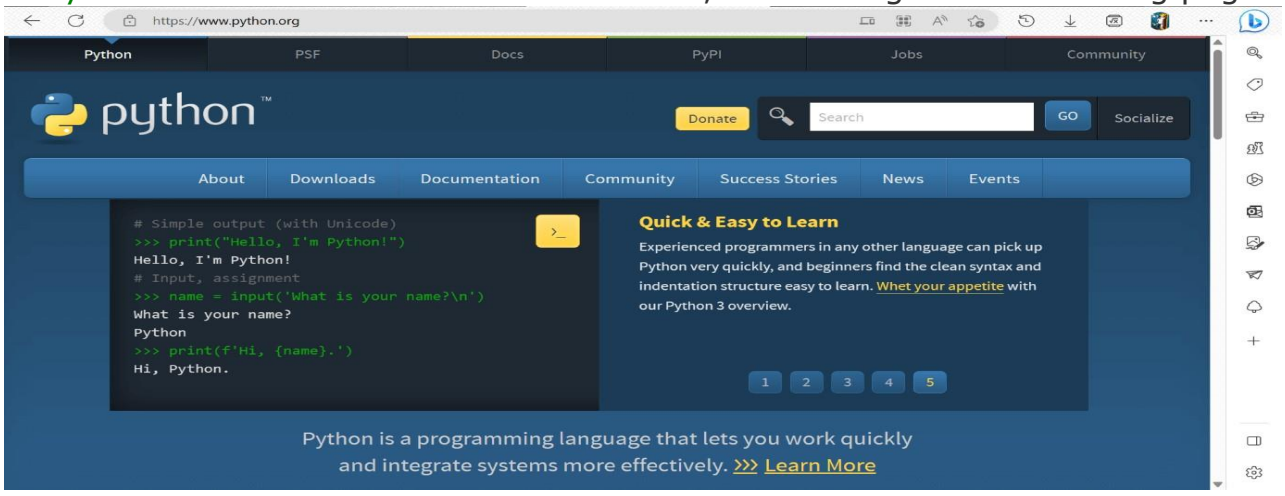
Features of Python:

- **Easy to use and Read -** Python's syntax is clear and easy to read, making it an ideal language for both beginners and experienced programmers. This simplicity can lead to faster development and reduce the chances of errors.
- **Dynamically Typed** - The data types of variables are determined during runtime. We do not need to specify the data type of a variable during writing codes.
- **High-level** - High-level language means human readable code.
- **Compiled and Interpreted** - Python code first gets compiled into bytecode, and then interpreted line by line. When we download the Python in our system form org we download the default implement of Python known as CPython. CPython is considered to be Complied and Interpreted both.
- **Garbage Collected** - Memory allocation and de-allocation are automatically managed. Programmers do not specifically need to manage the memory.
- **Purely Object-Oriented** - It refers to everything as an object, including numbers and strings.
- **Cross-platform Compatibility** - Python can be easily installed on Windows, macOS, and various Linux distributions, allowing developers to create software that runs across different operating systems.
- **Rich Standard Library** - Python comes with several standard libraries that provide ready-to-use modules and functions for various tasks, ranging from **web development** and **data manipulation** to **machine learning** and **networking**.
- **Open Source** - Python is an open-source, cost-free programming language. It is utilized in several sectors and disciplines as a result.

PYTHON APPLICATIONS:

Installation on Windows

Visit the link ***https://www.python.org*** to download the latest release of Python. In this process, we will install Python **3.11.3** on our Windows operating system. When we click on the above link, it will bring us the following page



-

Step 1: Select the Python version to download

Step 2: Download the required Python setup file

Step 3: Double click on the downloaded setup file to install Python

Python Variables

A variable is the name given to a memory location. A value-holding Python variable is also known as an identifier.

Since Python is an infer language that is smart enough to determine the type of a variable, we do not need to specify its type in Python.

Variable names must begin with a letter or an underscore, but they can be a group of both letters and digits.

The name of the variable should be written in lowercase. Both Rahul and rahul are distinct variables.

Identifier Naming

Identifiers are things like variables. An Identifier is utilized to recognize the literals utilized in the program. The standards to name an identifier are given underneath.

- The variable's first character must be an underscore or alphabet (_).
- Every one of the characters with the exception of the main person might be a letter set of lower-case(a-z), capitalized (A-Z), highlight, or digit (0-9).
- White space and special characters (!, @, #, %, etc.) are not allowed in the identifier name. ^, &, *).
- Identifier name should not be like any watchword characterized in the language.
- Names of identifiers are case-sensitive; for instance, my name, and MyName isn't something very similar.
- Examples of valid identifiers: a123, _n, n_9, etc. ○ Examples of invalid identifiers: 1a, n%4, n 9, etc.

Declaring Variable and Assigning Values

- Python doesn't tie us to pronounce a variable prior to involving it in the application. It permits us to make a variable at the necessary time.

•

- In Python, we don't have to explicitly declare variables. The variable is declared automatically whenever a value is added to it. - The equal (=) operator is utilized to assign worth to a variable.

Python Data Types

Every value has a datatype, and variables can hold values. Python is a powerfully composed language; consequently, we don't have to characterize the sort of variable while announcing it. The interpreter binds the value implicitly to its type.

```
a = 5
```

We did not specify the type of the variable a, which has the value five from an integer. The Python interpreter will automatically interpret the variable as an integer. We can verify the type of the program-used variable thanks to Python. The type() function in Python returns the type of the passed variable.

Consider the following illustration when defining and verifying the values of various data types.

```
1. a=10
2. b="Hi Python"
3. c = 10.5
4. print(type(a))
5. print(type(b))
6. print(type(c))
```

Output:

```
<type 'int'>
<type 'str'>
<type 'float'>
```

Python keywords:

Every scripting language has designated words or keywords, with particular definitions and usage guidelines. Python is no exception. The fundamental constituent elements of any Python program are Python keywords.

Example:

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while

Python Operators:

In general, **Operators** are the symbols used to perform a specific operation on different values and variables. These values and variables are considered as the **Operands,** on which the operator is applied. Operators serve as the foundation upon which logic is constructed in a program in a particular programming language.

In every programming language, some operators perform several tasks.

Different Types of Operators in Python

Same as other languages, Python also has some operators, and these are given below

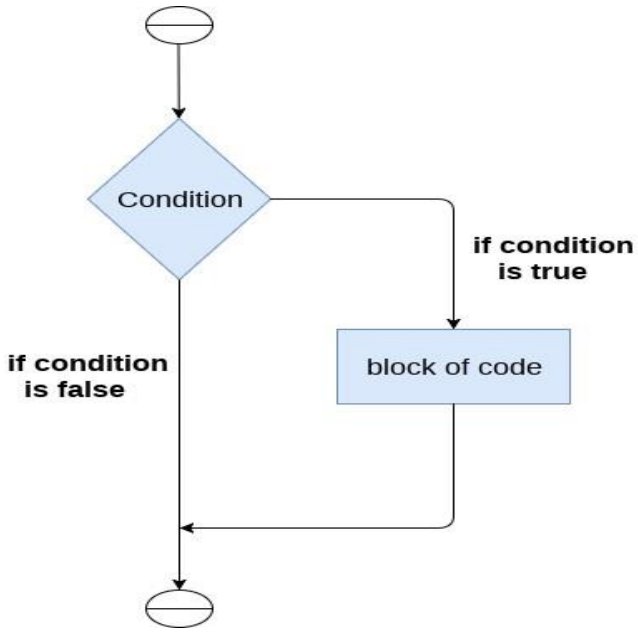
-

1. Arithmetic Operators
2. Comparison Operators
3. Assignment Operators
4. Logical Operators
5. Bitwise Operators
6. Membership Operators
7. Identity Operators

The if statement

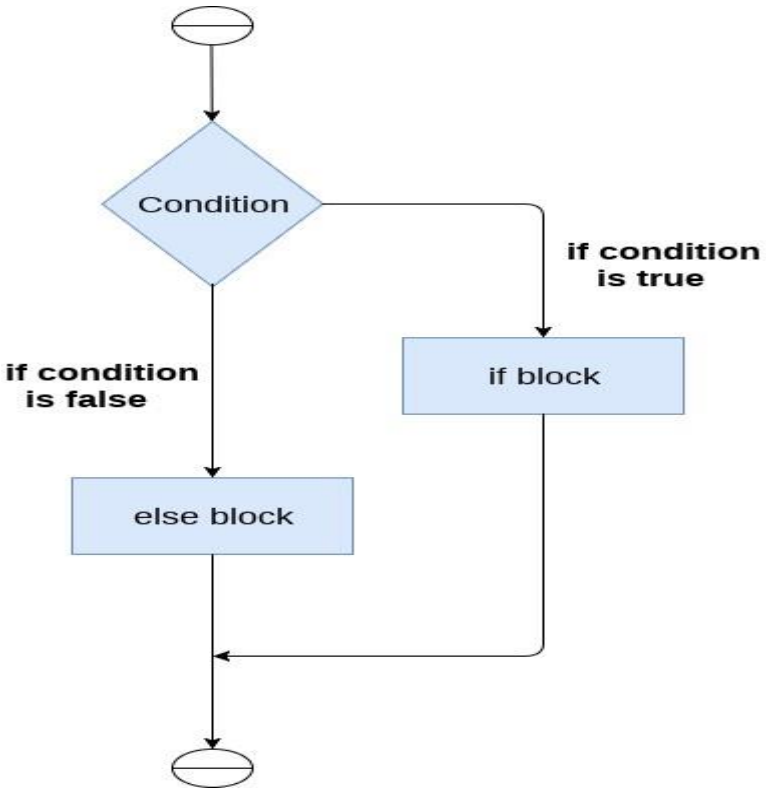
The if statement is used to test a particular condition and if the condition is true, it executes a block of code known as if-block. The condition of if statement can be any valid logical expression which can be either evaluated to true or false

•



The if-else statement

The if-else statement provides an else block combined with the if statement which is executed in the false case of the condition.
If the condition is true, then the if-block is executed. Otherwise, the else-block is executed.

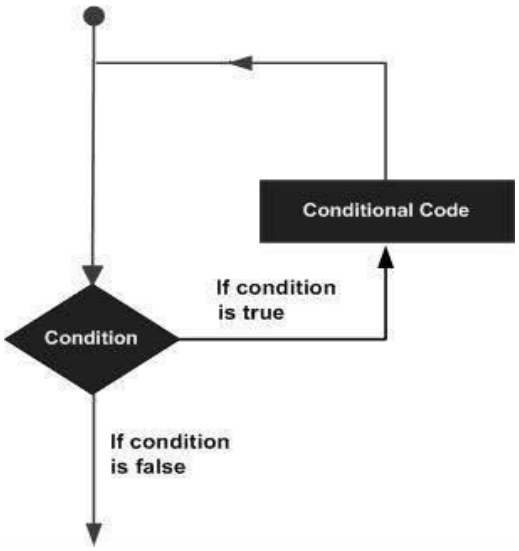


•

Loops:

Python loops allow us to execute a statement or group of statements multiple times. In general, statements are executed sequentially: The first statement in a function is executed first, followed by the second, and so on. There may be a situation when you need to execute a block of code several number of times.

Flowchart of loop is as follows



Python String:

Python string is the collection of the characters surrounded by single quotes, double quotes, or triple quotes. The computer does not understand the characters; internally, it stores manipulated character as the combination of the 0's and 1's. Each character is encoded in the ASCII or Unicode character. So we can say that Python strings are also called the collection of Unicode characters.

Consider the following example in Python to create a string.

Syntax:

```
str = "Hi Python !"
```

Here, if we check the type of the variable **str** using a Python script **print**(type(str)), then it will **print** a string (str).

•

In Python, strings are treated as the sequence of characters, which means that Python doesn't support the character data-type; instead, a single character written as 'p' is treated as the string of length 1.

Creating String in Python

We can create a string by enclosing the characters in single-quotes or double-quotes. Python also provides triple-quotes to represent the string, but it is generally used for multiline string or **docstrings**.

```
#Using single quotes
str1 = 'Hello Python'
print(str1)
#Using double quotes
str2 = "Hello Python"
print(str2)
#Using triple quotes
str3 = ''''' Triple quotes are generally used for represent the multiline or docstring
'''
print(str3)
```

Output:

Hello Python

Hello Python

Triple quotes are generally used for

represent the multiline or docstring

Python List

In Python, the sequence of various data types is stored in a list. A list is a collection of different kinds of values or items. Since Python lists are mutable, we can change their elements after forming. The comma (,) and the square brackets [enclose the List's items] serve as separators.

Lists written in Python are identical to dynamically scaled arrays defined in other languages, such as Array List in Java and Vector in C++. A list is a collection of items separated by commas and denoted by the symbol [].

List Example

```
list1 = [1, 2, "Python", "Program", 15.9]
list2 = ["Amy", "Ryan", "Henry", "Emma"]
```

```
3. print(list1)
4. print(list2)
5. print(type(list1))
6. print(type(list2))
```

Output:

[1, 2, 'Python', 'Program', 15.9]

['Amy', 'Ryan', 'Henry', 'Emma']

< class ' list ' >

< class ' list ' >

Characteristics of Lists

The characteristics of the List are as follows:

- The lists are in order.
- The list element can be accessed via the index.
- The mutable type of List is
- The rundowns are changeable sorts.
- The number of various elements can be stored in a list.

Tuples:

A comma-separated group of items is called a Python triple. The ordering, settled items, and reiterations of a tuple are to some degree like those of a rundown, but in contrast to a rundown, a tuple is unchanging.

The main difference between the two is that we cannot alter the components of a tuple once they have been assigned. On the other hand, we can edit the contents of a list.

Example

("Suzuki", "Audi", "BMW"," Skoda ") is a tuple.

Features of Python Tuple

- Tuples are an immutable data type, meaning their elements cannot be changed after they are generated.
- Each element in a tuple has a specific order that will never change because tuples are ordered sequences.

Python Set

A Python set is the collection of the unordered items. Each element in the set must be unique, immutable, and the sets remove the duplicate elements. Sets are mutable which means we can modify it after its creation.

Unlike other collections in Python, there is no index attached to the elements of the set, i.e., we cannot directly access any element of the set by the index. However, we can print them all together, or we can get the list of elements by looping through the set.

Creating a set

The set can be created by enclosing the comma-separated immutable items with the curly braces {}. Python also provides the set() method, which can be used to create the set by the passed sequence.

Example:

```
1. Days = {"Monday", "Tuesday", "Wednesday", "Thursday", "Friday",
   "Saturday", "S unday"}
2. print(Days)
3. print(type(Days))
4. print("looping through the set elements ... ")
5. for i in Days:
6. print(i)
```

Python Dictionary

Dictionaries are a useful data structure for storing data in Python because they are capable of imitating real-world data arrangements where a certain value exists for a given key.

The data is stored as key-value pairs using a Python dictionary.

- This data structure is mutable
- The components of dictionary were made using keys and values.
- Keys must only have one component.
- Values can be of any type, including integer, list, and tuple.

•

A dictionary is, in other words, a group of key-value pairs, where the values can be any Python object. The keys, in contrast, are immutable Python objects, such as strings, tuples, or numbers. Dictionary entries are ordered as of Python version 3.7.

In Python 3.6 and before, dictionaries are generally unordered.

Creating the Dictionary

Curly brackets are the simplest way to generate a Python dictionary, although there are other approaches as well. With many key-value pairs surrounded in curly brackets and a colon separating each key from its value, the dictionary can be built. (:). The following provides the syntax for defining the dictionary.

Syntax:

Dict = {"Name": "Gayle", "Age": 25}

In the above dictionary **Dict**, The keys **Name** and **Age** are the strings which comes under the category of an immutable object.

Example:

```
Employee = {"Name": "Johnny", "Age": 32, "salary":26000,"Company":"^TCS"} print(type(Employee)) print("printing Employee data .... ") print(Employee)
```

Python Functions

A collection of related assertions that carry out a mathematical, analytical, or evaluative operation is known as a function. An assortment of proclamations called Python Capabilities returns the specific errand. Python functions are necessary for intermediate-level programming and are easy to define. Function names meet the same standards as variable names do. The objective is to define a function and group-specific frequently performed actions. Instead of repeatedly creating the same code block for various input variables, we can call the function and reuse the code it contains with different variables.

•

Client-characterized and worked-in capabilities are the two primary classes of capabilities in Python. It aids in maintaining the program's uniqueness, conciseness, and structure.

Advantages of Python Functions

Pause We can stop a program from repeatedly using the same code block by including functions.

- Once defined, Python functions can be called multiple times and from any location in a program.
- Our Python program can be broken up into numerous, easy-to-follow functions if it is significant.
- The ability to return as many outputs as we want using a variety of arguments is one of Python's most significant achievements.
- However, Python programs have always incurred overhead when calling functions.

However, calling functions has always been overhead in a Python program. **Syntax def** function_name(parameters):

code block

Python Lambda Functions

This tutorial will study anonymous, commonly called lambda functions in Python. A lambda function can take n number of arguments at a time. But it returns only one argument at a time. We will understand what they are, how to execute them, and their syntax.

What are Lambda Functions in Python?

Lambda Functions in Python are anonymous functions, implying they don't have a name. The def keyword is needed to create a typical function in Python, as we already know. We can also use the lambda keyword in Python to define an unnamed function.

Syntax

lambda arguments: expression

This function accepts any count of inputs but only evaluates and returns one expression. That means it takes many inputs but returns only one output.

•

What is Modular Programming?

Modular programming is the practice of segmenting a single, complicated coding task into multiple, simpler, easier-to-manage sub-tasks. We call these subtasks modules. Therefore, we can build a bigger program by assembling different modules that act like building blocks.

Modularizing our code in a big application has a lot of benefits.

Simplification: A module often concentrates on one comparatively small area of the overall problem instead of the full task. We will have a more manageable design problem to think about if we are only concentrating on one module. Program development is now simpler and much less vulnerable to mistakes.

Flexibility: Modules are frequently used to establish conceptual separations between various problem areas. It is less likely that changes to one module would influence other portions of the program if modules are constructed in a fashion that reduces interconnectedness. (We might even be capable of editing a module despite being familiar with the program beyond it.) It increases the likelihood that a group of numerous developers will be able to collaborate on a big project.

Reusability: Functions created in a particular module may be readily accessed by different sections of the assignment (through a suitably established api). As a result, duplicate code is no longer necessary.

Scope: Modules often declare a distinct namespace to prevent identifier clashes in various parts of a program.

In Python, modularization of the code is encouraged through the use of functions, modules, and packages.

What are Modules in Python?

A document with definitions of functions and various statements written in Python is called a Python module.

In Python, we can define a module in one of 3 ways:
- Python itself allows for the creation of modules.

- Similar to the re (regular expression) module, a module can be primarily written in C programming language and then dynamically inserted at run-time.
- A built-in module, such as the itertools module, is inherently included in the interpreter.

A module is a file containing Python code, definitions of functions, statements, or classes. An example_module.py file is a module we will create and whose name is example_module.

We employ modules to divide complicated programs into smaller, more understandable pieces. Modules also allow for the reuse of code.

Rather than duplicating their definitions into several applications, we may define our most frequently used functions in a separate module and then import the complete module.

Example:

```
1. # Here, we are creating a simple Python program to show how to
   create a module
2. # defining a function in the module to reuse it
3. def square( number ):
4. # here, the above function will square the number passed as the
   input
5. result = number ** 2
6. return result      # here, we are returning the result of the
   function
```

Here, a module called example_module contains the definition of the function square(). The function returns the square of a given number.

How to Import Modules in Python?

In Python, we may import functions from one module into our program, or as we say into, another module.

For this, we make use of the import Python keyword. In the Python window, we add the next to import keyword, the name of the module we need to import. We will import the module we defined earlier example_module.

•

Syntax: import

example_module

The functions that we defined in the example_module are not immediately imported into the present program. Only the name of the module, i.e., example_ module, is imported here.

We may use the dot operator to use the functions using the module name. For instance:

Example:

```
1. # here, we are calling the module square method and passing
   the value 4
2. result = example_module.square(  4  )
3. print("By using the module square of number is: ", result )
```

Output:

By using the module square of number is:

What is an Exception?

An exception in Python is an incident that happens while executing a program that causes the regular course of the program's commands to be disrupted. When a Python code comes across a condition it can't handle, it raises an exception. An object in Python that describes an error is called an exception. When a Python code throws an exception, it has two options: handle the exception immediately or stop and quit.

Python OOPs Concepts

Like other general-purpose programming languages, Python is also an object-oriented language since its beginning. It allows us to develop applications using an ObjectOriented approach. In Python, we can easily create and use classes and objects. An object-oriented paradigm is to design the program using classes and objects. The object is related to real-word entities such as book, house, pencil, etc. The oops concept focuses on writing the reusable code. It is a widespread technique to solve the problem by creating objects.

Major principles of object-oriented programming system are given below.

-

- Class
  Object
- Method
- Inheritance
- Polymorphism
- Data Abstraction
- Encapsulation

Class

The class can be defined as a collection of objects. It is a logical entity that has some specific attributes and methods. For example: if you have an employee class, then it should contain an attribute and method, i.e. an email id, name, age, salary, etc. **Syntax** **class** ClassName: <statement-1>

.

.

<statement-N>

Object

The object is an entity that has state and behavior. It may be any real-world object like the mouse, keyboard, chair, table, pen, etc.

Everything in Python is an object, and almost everything has attributes and methods. All functions have a built-in attribute __doc__, which returns the docstring defined in the function source code.

When we define a class, it needs to create an object to allocate the memory.

Example:

```
1. class car:
2. def __init__(self,modelname, year):
3. self.modelname = modelname
4. self.year = year
5. def display(self):
6. print(self.modelname,self.year)
7. c1 = car("Toyota", 2016)
8. c1.display()
```

-

Method

The method is a function that is associated with an object. In Python, a method is not unique to class instances. Any object type can have methods.

Inheritance

Inheritance is the most important aspect of object-oriented programming, which simulates the real-world concept of inheritance. It specifies that the child object acquires all the properties and behaviors of the parent object.

By using inheritance, we can create a class which uses all the properties and behavior of another class. The new class is known as a derived class or child class, and the one whose properties are acquired is known as a base class or parent class.

It provides the re-usability of the code.

Polymorphism

Polymorphism contains two words "poly" and "morphs". Poly means many, and morph means shape. By polymorphism, we understand that one task can be performed in different ways. For example - you have a class animal, and all animals speak. But they speak differently. Here, the "speak" behavior is polymorphic in a sense and depends on the animal. So, the abstract "animal" concept does not actually "speak", but specific animals (like dogs and cats) have a concrete implementation of the action "speak".

Encapsulation

Encapsulation is also an essential aspect of object-oriented programming. It is used to restrict access to methods and variables. In encapsulation, code and data are wrapped together within a single unit from being modified by accident.

Data Abstraction

Data abstraction and encapsulation both are often used as synonyms. Both are nearly synonyms because data abstraction is achieved through encapsulation.

Abstraction is used to hide internal details and show only functionalities. Abstracting something means to give names to things so that the name captures the core of what a function or a whole program does.

Python Constructor

A constructor is a special type of method (function) which is used to initialize the instance members of the class.

•

In C++ or Java, the constructor has the same name as its class, but it treats constructor differently in Python. It is used to create an object.

Constructors can be of two types.

1. Parameterized Constructor
2. Non-parameterized Constructor

Constructor definition is executed when we create the object of this class. Constructors also verify that there are enough resources for the object to perform any start-up task.

Creating the constructor in python

In Python, the method the **__init__()** simulates the constructor of the class. This method is called when the class is instantiated. It accepts the **self**-keyword as a first argument which allows accessing the attributes or method of the class.

We can pass any number of arguments at the time of creating the class object, depending upon the **__init__()** definition. It is mostly used to initialize the class attributes. Every class must have a constructor, even if it simply relies on the default constructor.

Consider the following example to initialize the **Employee** class attributes.

Example

```
1.  class Employee:
2.      def __init__(self, name, id):
3.          self.id = id   4. self.name = name
5.      def display(self):
6.          print("ID: %d \nName: %s" % (self.id, self.name))
7.  emp1 = Employee("John", 101)
8.  emp2 = Employee("David", 102)
9.  # accessing display() method to print employee 1 information
10.     emp1.display()
11.     # accessing display() method to print employee 2 information
12.     emp2.display()
```

Output:

ID: 101

Name: John

ID: 102

Name: David

Counting the number of objects of a class

The constructor is called automatically when we create the object of the class

Example

```
1.  class Student:
2.      count = 0
3.      def __init__(self):
4.          Student.count = Student.count + 1
5.  s1=Student()
6.  s2=Student()
7.  s3=Student()
8.  print("The number of students:",Student.count)
```

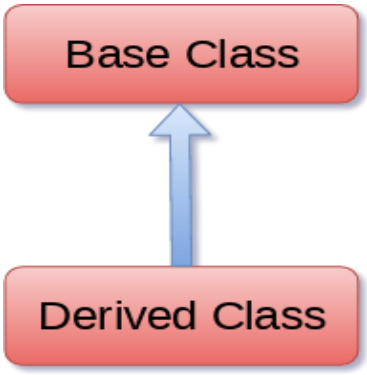
output: The number of students : 3

Python Inheritance

Inheritance is an important aspect of the object-oriented paradigm. Inheritance provides code reusability to the program because we can use an existing class to create a new class instead of creating it from scratch.

In inheritance, the child class acquires the properties and can access all the data members and functions defined in the parent class. A child class can also provide its specific implementation to the functions of the parent class

In python, a derived class can inherit base class by just mentioning the base in the bracket after the derived class name. Consider the following syntax to inherit a base class into the derived class.



•

Syntax class derived-**class**(base **class**):

<**class**-suite>

A class can inherit multiple classes by mentioning all of them inside the bracket. Consider the following syntax. **Syntax class** derive-**class**(<base **class** 1>, <base **class** 2>, ... <base **class** n>): <**class** - suite>

Example 1

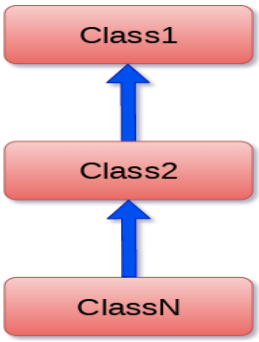
```
1.  class Animal:
2.      def speak(self):
3.          print("Animal Speaking")
4.  #child class Dog inherits the base class Animal
5.  class Dog(Animal):
6.      def bark(self):
7.          print("dog barking")
8.  d = Dog()
9.  d.bark()
10. d.speak()
```

Output: *dog barking*

Animal Speaking

Python Multi-Level inheritance

Multi-Level inheritance is possible in python like other object-oriented languages. Multi-level inheritance is archived when a derived class inherits another derived class. There is no limit on the number of levels up to which, the multi-level inheritance is archived in python.



•

The syntax of multi-level inheritance is given below.

Syntax

```
class class1:
<class-suite>
class class2(class1):
<class suite>
class class3(class2):
<class suite>
.
.
```

Example

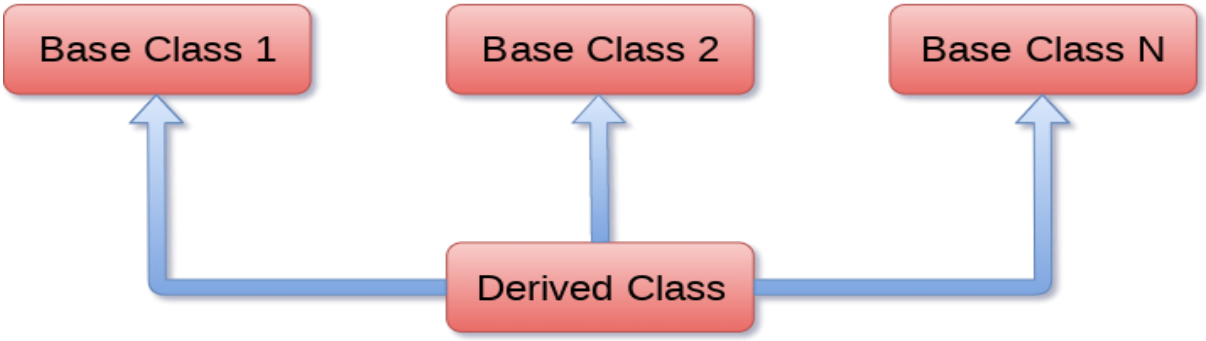
```
class Animal:
def speak(self):
print("Animal Speaking")
#The child class Dog inherits the base class Animal
class Dog(Animal):
def bark(self):
print("dog barking")
#The child class Dogchild inherits another child class Dog
class DogChild(Dog):
def eat(self):
print("Eating bread...")
d = DogChild()
d.bark()
d.speak()
d.eat()
```

Output: *dog barking Animal Speaking Eating bread...*

Python Multiple inheritance

Python provides us the flexibility to inherit multiple base classes in the child class.

•



The syntax to perform multiple inheritance is given below.

Syntax

```
1. class Base1:
2. <class-suite>

3. class Base2:
4. <class-suite>
5. .
6. .
7. .
8. class BaseN:
9. <class-suite>

10.     class Derived(Base1, Base2, ..... BaseN):
11.     <class-suite>
```

Example

```
1. class Calculation1:
2. def Summation(self,a,b):
3. return a+b;
4. class Calculation2:
5. def Multiplication(self,a,b):
6. return a*b;
7. class Derived(Calculation1,Calculation2):
8. def Divide(self,a,b):
9. return a/b;
d = Derived()
```

•

print(d.Summation(10,20)) **print**(d.Multiplication(10,20))
print(d.Divide(10,20)) **Output:**

30

200

0.5

The subclass(sub,sup) method

The subclass(sub, sup) method is used to check the relationships between the specified classes. It returns true if the first class is the subclass of the second class, and false otherwise.

Example

```
1. class Calculation1:
2. def Summation(self,a,b):
3. return a+b;
4. class Calculation2:
5. def Multiplication(self,a,b):
6. return a*b;
7. class Derived(Calculation1,Calculation2):
8. def Divide(self,a,b):
9. return a/b;
```

```
d = Derived()
print(issubclass(Derived,Calculation2))
print(issubclass(Calculation1,Calculation2))
```

Output:

True

False

Method Overriding

We can provide some specific implementation of the parent class method in our child class. When the parent class method is defined in the child class with some specific implementation, then the concept is called method overriding. We may need to perform method overriding in the scenario where the different definition of a parent class method is needed in the child class.

•

Example

```
1. class Animal:
2. def speak(self):
3. print("speaking")
4. class Dog(Animal):
5. def speak(self):
6. print("Barking")
7. d = Dog()
8. d.speak()
```

Output:

Barking

Data abstraction in python

Abstraction is an important aspect of object-oriented programming. In python, we can also perform data hiding by adding the double underscore (___) as a prefix to the attribute which is to be hidden. After this, the attribute will not be visible outside of the class through the object.

Example

```
1.  class Employee:
2.  __count = 0;
3.  def __init__(self):
4.  Employee.__count = Employee.__count+1
5.  def display(self):
6.  print("The number of employees",Employee.__count)
7.  emp = Employee()   8. emp2 = Employee()
9.  try:
10. print(emp.__count)
11. finally:
12. emp.display()
```

Output:

The number of employees 2

AttributeError: 'Employee' object has no attribute '__count'

•

DJANGO

Django is a web application framework written in Python programming language. It is based on MVT (Model View Template) design pattern. The Django is very demanding due to its rapid development feature. It takes less time to build application after collecting client requirement. This framework uses a famous tag line:

The web framework for perfectionists with deadlines.

By using Django, we can build web applications in very less time. Django is designed in such a manner that it handles much of configure things automatically, so we can focus on application development only.

Features of Django

- Rapid Development
- Secure
- Scalable
- Fully loaded
- Versatile
- Open Source
- Vast and Supported Community

Rapid Development

Django was designed with the intention to make a framework which takes less time to build web application. The project implementation phase is a very time taken but Django creates it rapidly.

Secure

Django takes security seriously and helps developers to avoid many common security mistakes, such as SQL injection, cross-site scripting, cross-site request forgery etc. Its user authentication system provides a secure way to manage user accounts and passwords.

Scalable

Django is scalable in nature and has ability to quickly and flexibly switch from small to large scale application project.

Fully loaded

Django includes various helping task modules and libraries which can be used to handle common Web development tasks. Django takes care of user authentication, content administration, site maps, RSS feeds etc.

Versatile

Django is versatile in nature which allows it to build applications for different-different domains. Now a days, Companies are using Django to build various types of applications like: content management systems, social networks sites or scientific computing platforms etc.

Open Source

Django is an open source web application framework. It is publicly available without cost. It can be downloaded with source code from the public repository. Open source reduces the total cost of the application development.

Vast and Supported Community

Django is an one of the most popular web framework. It has widely supportive community and channels to share and connect.

DJANGO PROJECT:

To create a Django project, we can use the following command. *projectname* is the name of Django application.

$ django-admin startproject projectname

Packages and Files

A Django project contains the following packages and files. The outer directory is just a container for the application. We can rename it further.

- **manage.py:** It is a command-line utility which allows us to interact with the project in various ways and also used to manage an application that we will see later on in this tutorial.
- A directory (djangpapp) located inside, is the actual application package name. Its name is the Python package name which we'll need to use to import module inside the application.

•

- **__init__.py:** It is an empty file that tells to the Python that this directory should be considered as a Python package.
- **settings.py:** This file is used to configure application settings such as database connection, static files linking etc.
- **urls.py:** This file contains the listed URLs of the application. In this file, we can mention the URLs and corresponding actions to perform the task and display the view.
- **wsgi.py:** It is an entry-point for WSGI-compatible web servers to serve Django project.

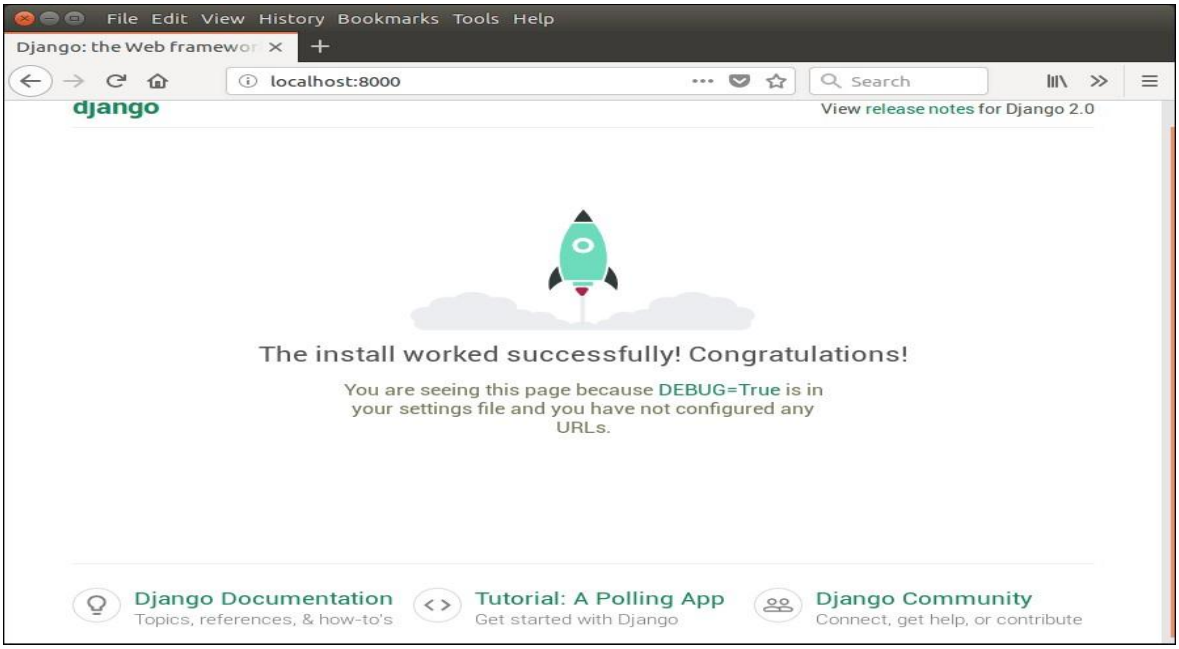
Initially, this project is a default draft which contains all the required files and folders.

Running the Django Project

Django project has a built-in development server which is used to run application instantly without any external web server. It means we don't need of Apache or another web server to run the application in development mode.

To run the application, we can use the following command.

$ python3 manage.py runserver



Django Virtual Environment Setup

The virtual environment is an environment which is used by Django to execute an application. It is recommended to create and execute a Django application in a separate environment. Python provides a tool **virtualenv** to create an isolated Python environment. We will use this tool to create a virtual environment for our Django application.

To set up a virtual environment, use the following steps.

Install Package

First, install **python3-venv** package by using the following command.

$ apt-get install python3-venv

2. Create a Directory

$ mkdir djangoenv

After it, change directory to the newly created directory by using the **cd djangoenv.**

3. Create Virtual Environment

$ python3 -m venv djangoenv

4. Activate Virtual Environment

After creating a virtual environment, activate it by using the following command. $ source djangoenv/bin/activate

Till here, the virtual environment has started. Now, we can use it to create Django application.

Install Django

Install Django in the virtual environment. To install Django, use the following command.

$ pip install django

Django Admin Interface

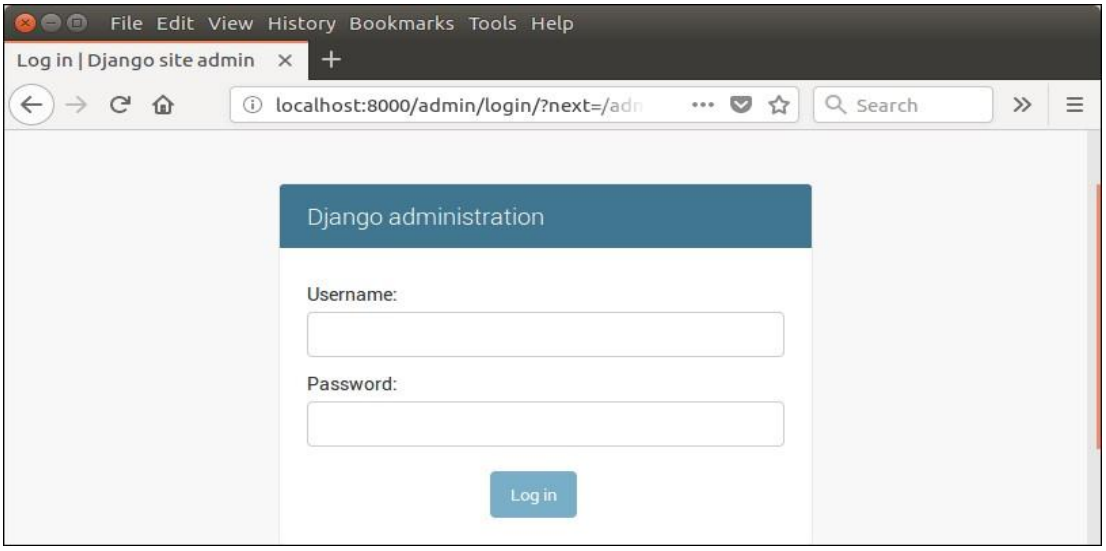
Django provides a built-in admin module which can be used to perform CRUD operations on the models. It reads metadata from the model to provide a quick interface where the user can manage the content of the application.

This is a built-in module and designed to perform admin related tasks to the user.

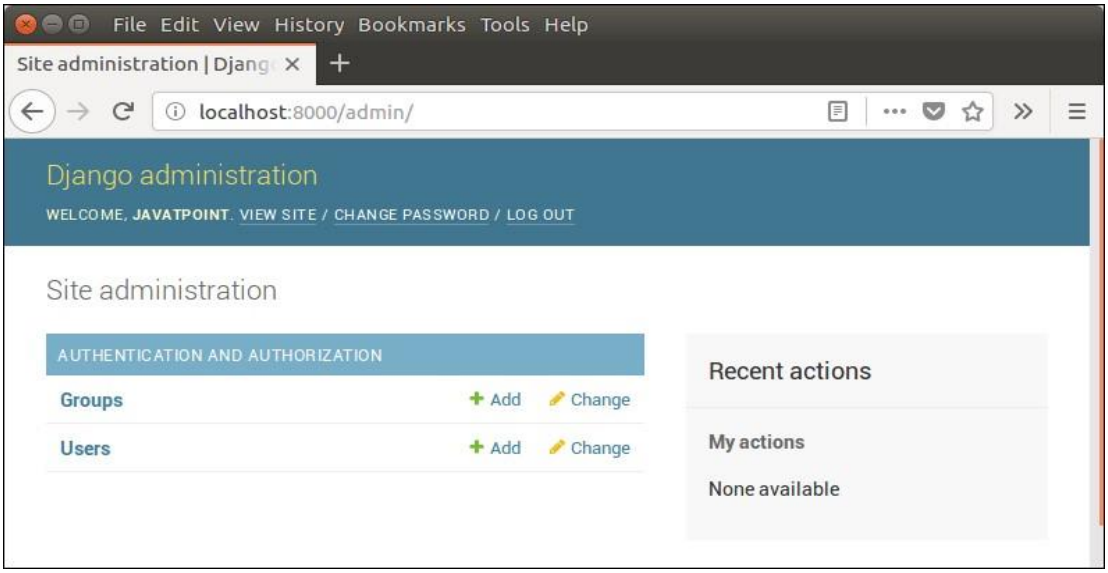
•

Let's see how to activate and use Django's admin module (interface).

The admin app **(django.contrib.admin)** is enabled by default and already added into INSTALLED_APPS section of the settings file.



After login



Django App

In the previous topics, we have seen a procedure to create a Django project. Now, in this topic, we will create app inside the created project.

Django application consists of project and app, it also generates an automatic base directory for the app, so we can focus on writing code (business logic) rather than creating app directories.

•

The difference between a project and app is, a project is a collection of configuration files and apps whereas the app is a web application which is written to perform business logic.

Creating an App

To create an app, we can use the following command.

```
$ python3 manage.py startapp appname
```

Example:

/ views.py

```
1. from django.shortcuts import render
2. # Create your views here.
3. from django.http import HttpResponse
4. def hello(request):
5. return HttpResponse("<h2>Hello, Welcome to Django!</h2>")
```

// urls.py

```
1. from django.contrib import admin
2. from django.urls import path
3. from myapp import views
4. urlpatterns = [
5. path('admin/', admin.site.urls),
6. path('hello/', views.hello),
7. ]
```

We have made changes in two files of the application. Now, let's run the it by using the following command. This command will start the server at port 8000. Run the Application

```
$ python3 manage.py runserver
```

Open any web browser and enter the URL **localhost:8000/hello**. It will show the output given below.

•

 # Console App – 1 (Quiz app)

```
score = 0
print ("Welcome to QUIZ app") play = input("Do you want to play (yes/no) : ")
if play.lower() != 'yes' :
    quit()
print('OK! let\'s play')
answer = input('CPU stands for : ')
if answer.lower() == 'central processing unit' :
    print('Correct')
    score = score + 1
else :
    print('Incorrect')
    print('correct answer is : central processing unit' )
proceed = input('Do you want to continue? (yes/no) ')
if proceed.lower() != 'yes' :
    print("Your score is : " , score)
    quit()
else :
    answer = input('Standard input device is : ')
    if answer.lower() == 'keyboard' :
        print('Correct')
        score = score + 1
```

•

```
    else :
        print('Incorrect')
        print('correct answer is : keyboard' )
proceed = input('Do you want to continue? (yes/no) ')
if proceed.lower() != 'yes' :
    print("Your score is : " , score)
    quit()
else :
    answer = input('Standard output device is : ')
if answer.lower() == 'monitor' :
    print('Correct')
    score = score + 1
else :
    print('Incorrect')
    print('correct answer is : monitor' )
    print("Your score is : " , score)
```

Output 1:

```
Welcome to QUIZ app
Do you want to play (yes/no) : yes
OK! let's play
CPU stands for : Central Processing Unit
Correct
Do you want to continue? (yes/no) no
Your score is :  1
```

Output 2:

```
Welcome to QUIZ app
Do you want to play (yes/no) : yes
OK! let's play
CPU stands for : Central Processing Unit
Correct
Do you want to continue? (yes/no) yes
Standard input device is : Keyboard
Correct
Do you want to continue? (yes/no) no
Your score is :  2
```

•

Output 3:

```
Welcome to  QUIZ  app
Do you want to play (yes/no) : yes
OK! let's play
CPU stands for : central processing unit
Correct
Do you want to continue? (yes/no) yes
Standard input device is : keyboard
Correct
Do you want to continue? (yes/no) yes
Standard output device is : monitor
Correct
Your score is :  3
```

Output 4:

```
Welcome to  QUIZ  app
Do you want to play (yes/no) : yes
OK! let's play
CPU stands for : cpu
Incorrect
correct answer is : central processing unit
Do you want to continue? (yes/no) yes
Standard input device is : keyboard
Correct
Do you want to continue? (yes/no) yes
Standard output device is : monitor
Correct
Your score is :  2
```

 # Console App – 2 (ATM Transactions)

```
print ('Welcome to  ATM  app')
pin = 1234
balance = 5000
def deposit():
global balance
 amount = int (input('Enter amount to deposit : '))
if amount % 100 != 0 :
      print('Amount must be multiple of 100 : ')
```

•

```
    else:
        balance = balance + amount
        print('Current balance is : ' , balance)
def withdraw():
    global balance
    amount = int (input('Enter amount to withdraw : '))
    if amount > balance :
        print('Insufficient funds  ')
    else:
        if amount % 100 != 0 :
            print('Amount must be multiple of 100 : ')
        else:
            balance = balance - amount
            print('Current balance is : ' , balance)
def checkBalance():
    print ('Balance is : ' ,balance)
def changePin():
    global pin
    newpin = int (input('Enter new atm pin : '))
    pin = newpin
    print('Your ATM is changed successfully...')
    doTransaction()

def doTransaction():
    atmpin = int (input('Enter ATM Pin : '))
    if atmpin != pin :
        quit()
    else:
        while True:
            print('1. Deposit : ')
            print('2. Withdraw : ')
            print('3. Balance : ')
```

```
                print('4. Chang pin : ')
                print('5. Exit : ')
               choice = int(input('Enter your choice : '))
                match choice:
                    case 1 : deposit()
                    case 2 : withdraw()
                    case 3 : checkBalance()
                    case 4 : changePin()
                    case 5 : quit()
                    case _ : print('invalid choice')

doTransaction()
```

Output 1:

```
Welcome to HQL ATM app
Enter ATM Pin : 1222
You have entered wrong pin
```

Output 2:

```
Welcome to ATM app
Enter ATM Pin : 1234
1. Deposit :
2. Withdraw :
3. Balance :
4. Chang pin :
5. Exit :
Enter your choice : 3
Balance is :  5000
1. Deposit :
2. Withdraw :
3. Balance :
4. Chang pin :
5. Exit :
```

•

Enter your choice : 1

Enter amount to deposit : 2000

Current balance is : 7000

1. Deposit :
2. Withdraw :
3. Balance :
4. Chang pin :
5. Exit :

Enter your choice : 2

Enter amount to withdraw : 1000

Current balance is : 6000

1. Deposit :
2. Withdraw :
3. Balance :
4. Chang pin :
5. Exit :

Enter your choice : 4 Enter new atm pin : 1111

Your ATM is changed successfully...

Enter ATM Pin : 1111

1. Deposit :
2. Withdraw :
3. Balance :
4. Chang pin :
5. Exit :

Enter your choice : 3

Balance is : 6000

1. Deposit :
2. Withdraw :
3. Balance :
4. Chang pin :
5. Exit :

•

```
Enter your choice : 5
>>>
```

DJANGO CRUD APP (Employ management system)

Home.html

```
<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>Document</title>
    </head>
    <body>
        {% csrf_token %}
        <a href="/add/"> Add </a> <br><br>
        <table border="1">
        <thead>
        <tr>
            <th> empno </th>
            <th> empname </th>
            <th> email </th>
            <th> salary </th>
            <th colspan="2"> action </th>
        </tr>
        </thead>
        <tbody>
            {% for emp in employ %}
            <tr>
                <td> {{ emp.empno }} </td>
```

•

```
            <td> {{ emp.empname }} </td>
            <td> {{ emp.email }} </td>
            <td> {{ emp.salary }} </td>
            <td> <a href="/update/{{emp.empno}}"> Edit </a> </td>
            <td> <a href="/delete/{{emp.empno}}"> Delete </a> </td>
          </tr>
          {% endfor %}
        </tbody>
      </table>
  </body>
</html>
```

Models.py

```
from django.db import models
# Create your models here.
class employ(models.Model):
    empno = models.IntegerField()
    empname = models.CharField(max_length=20)
    email = models.CharField(max_length=30)
   salary = models.FloatField()
```

Views.py

```
from django.shortcuts import render, redirect
from crudapp.models import employ import pypyodbc
# Function to return SQL Server connection string
def connection():
    con = pypyodbc.connect(
        'Driver={SQL Server Native Client 11.0};'
        'Server=DESKTOP-BJNT3TS;'
        'Database=BVC_MCA;'
```

•

```
        'Trusted_Connection=yes;'
        )
    return con;
# Function to get all records from data base table
def getAllRecords(request):
     conn = connection()
     cursor = conn.cursor()
    cursor.execute("select * from employ")
     result = cursor.fetchall()
    return render(request, 'crudapp/home.html', {'employ': result}) # Function to
Add Record into database table


def addRecord(request):
      conn = connection()
      if (request.method == 'POST'):
           if request.POST.get('empno') and request.POST.get('empname') and
request.POST.get('email') and request.POST.get('salary'):
insertvalues=employ();
            insertvalues.empno=request.POST.get('empno')
insertvalues.empname=request.POST.get('empname')
insertvalues.email=request.POST.get('email')
insertvalues.salary=request.POST.get('salary')
cursor=conn.cursor()
            cursor.execute("Insert Into Employ Values
('"+insertvalues.empno+"','"+insertvalues.empname+"','"+insertvalues.email+"',
'"+i nsertvalues.salary+"')")
             cursor.commit()
             return redirect('/home')
         else:
              return render(request, 'crudapp/insertdata.html')
              else:
                   return render(request, 'crudapp/insertdata.html')
```

•

```
# Function to delete a record from database table


def deleteRecord(request, empno):
    print("empno is " , empno)
    conn = connection()
    cursor = conn.cursor()
    cursor.execute("delete from employ where empno= ?", (empno,))
    conn.commit()
    return redirect('/home/')
# Function to update a record


def updateRecord(request, empno):
        conn = connection()
        cursor = conn.cursor()
        if request.method == 'GET' :
            cursor.execute("select * from employ where empno=?",(empno,))
            result = cursor.fetchall()
            conn.close()
            return render(request, 'crudapp/updatedata.html', {'employ': result[0]})


        if (request.method == 'POST'):
            if  request.POST.get('empname') and request.POST.get('email') and
      request.POST.get('salary'):
                updatevalues=employ();
                updatevalues.empname=request.POST.get('empname')
                updatevalues.email=request.POST.get('email')
                updatevalues.salary=request.POST.get('salary')
                cursor=conn.cursor()
                cursor.execute("Update Employ set empname=?, email=?,
        salary=? where empno=?", (updatevalues.empname,
        updatevalues.email,updatevalues.salary, empno))
                cursor.commit()
```

•

```
                return redirect('/home')
            else:
                return render(request, 'crudapp/updatedata.html')
        else:
            return render(request, 'crudapp/updatedata.html')
```

updatedata.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title> update record </title>
</head>
<body>
    <form action="" method="POST">
        {% csrf_token %}
        empno : <input type="text" name="empno" value="{{employ.empno}}"
readonly /> <br>
        ename : <input type="text" name="empname"
value="{{employ.empname}}"
/> <br>
        email : <input type="text" name="email" value="{{employ.email}}"  />
<br>        salary : <input type="text" name="salary"
value="{{employ.salary}}" /> <br>        <input type="submit" value="Save"
/>
    </form>
</body>
</html>
```

-

Insertdata.html

```
<!DOCTYPE html> <html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
<title> add record </title>
</head>
<body>
  <form action="" method="POST">
      {% csrf_token %}
      empno : <input type="text" name="empno" /> <br>        ename :
<input type="text" name="empname" /> <br>        email : <input type="text"
name="email" /> <br>        salary : <input type="text" name="salary" />
<br>
      <input type="submit" value="Save" />
   </form>
</body>
</html>
```

Urls.py

```
"""
URL configuration for crudproject project.

The `urlpatterns` list routes URLs to views. For more information please see:
    https://docs.djangoproject.com/en/5.0/topics/http/urls/ Examples:
Function views
    1. Add an import:  from my_app import views
    2. Add a URL to urlpatterns:  path('', views.home, name='home')
```

•

Class-based views

1. Add an import: from other_app.views import Home
2. Add a URL to urlpatterns: path('', Home.as_view(), name='home')

Including another URLconf

1. Import the include() function: from django.urls import include, path
2. Add a URL to urlpatterns: path('blog/', include('blog.urls')) """

from django.contrib import admin from django.urls import path

from crudapp import views

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', views.getAllRecords),
    path('home/', views.getAllRecords),
    path('add/', views.addRecord),
    path('delete/<int:empno>/', views.deleteRecord),
    path('update/<int:empno>/', views.updateRecord)
]
```

Settings.py

"""

Django settings for crudproject project.

Generated by 'django-admin startproject' using Django 5.0.4.

For more information on this file, see

https://docs.djangoproject.com/en/5.0/topics/settings/

For the full list of settings and their values, see
https://docs.djangoproject.com/en/5.0/ref/settings/

"""

-

```
from pathlib import Path import os


# Build paths inside the project like this: BASE_DIR / 'subdir'.

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
TEMPLATE_DIR = os.path.join(BASE_DIR,'templates')


# Quick-start development settings - unsuitable for production

# See https://docs.djangoproject.com/en/5.0/howto/deployment/checklist/ #
SECURITY WARNING: keep the secret key used in production secret!

SECRET_KEY = 'django-insecure-h0l^ao%(1o_enux1p#3whptwqq-
buu(yy3$dy@vf=21ybz7kv'


# SECURITY WARNING: don't run with debug turned on in production!

DEBUG = True


ALLOWED_HOSTS = []


# Application definition


INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'crudapp',
]


MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
```

-

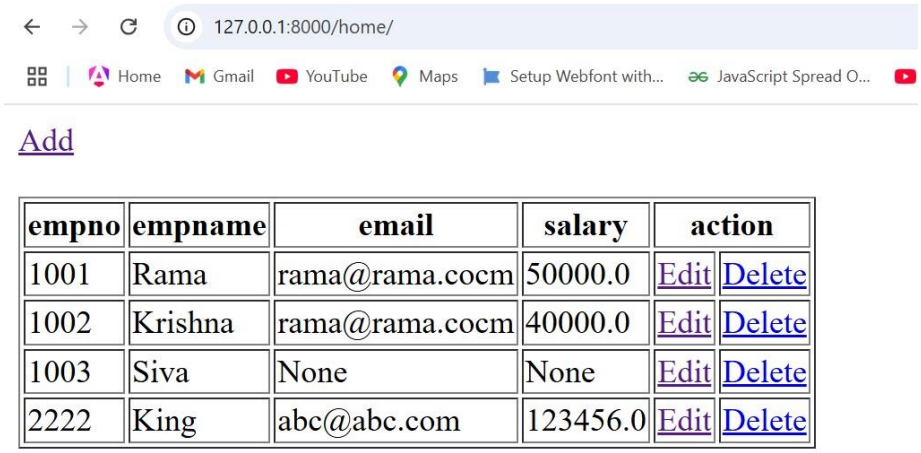
```
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'crudproject.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [TEMPLATE_DIR],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

WSGI_APPLICATION = 'crudproject.wsgi.application'
# Database
# https://docs.djangoproject.com/en/5.0/ref/settings/#databases
DATABASES = {
    'default': {
      # 'ENGINE': 'django.db.backends.sqlite3',
      •
```

```
        # 'NAME': BASE_DIR / 'db.sqlite3',
    }
}
# Password validation
# https://docs.djangoproject.com/en/5.0/ref/settings/#auth-password-validators
AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    { 'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]
# Internationalization
# https://docs.djangoproject.com/en/5.0/topics/i18n/
LANGUAGE_CODE = 'en-us'
TIME_ZONE = 'UTC'
USE_I18N = True
USE_TZ = True
# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/5.0/howto/static-files/
STATIC_URL = 'static/'
# Default primary key field type
```

-

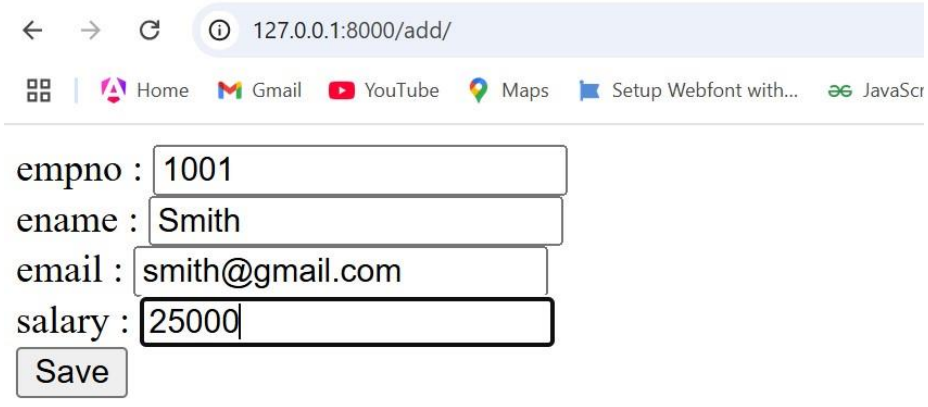
```
# https://docs.djangoproject.com/en/5.0/ref/settings/#default-auto-field

DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
```

Output:



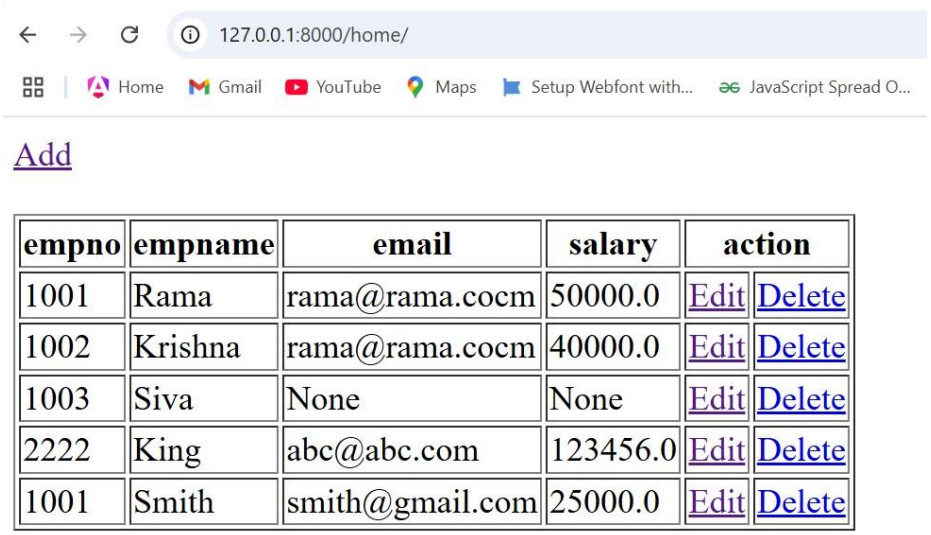
When user clicks on Add button then it will redirect to add page

Add employ data and click on Save button



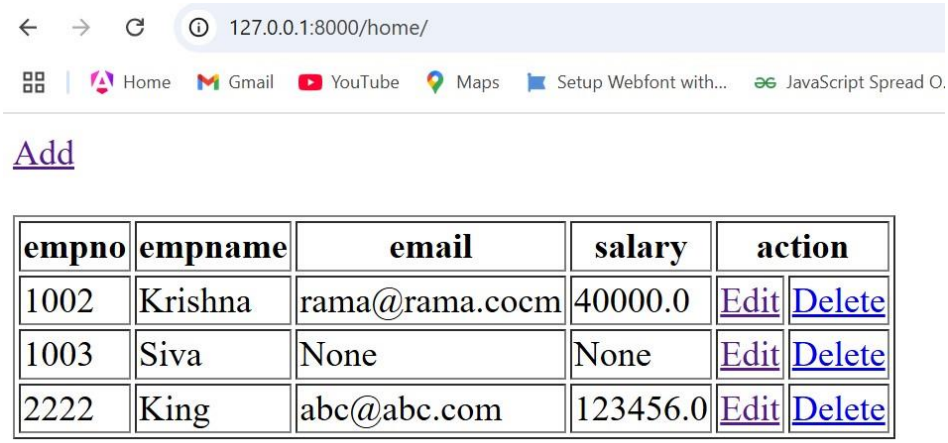
After clicking on Save button it will redirect to home page again with latest data

-



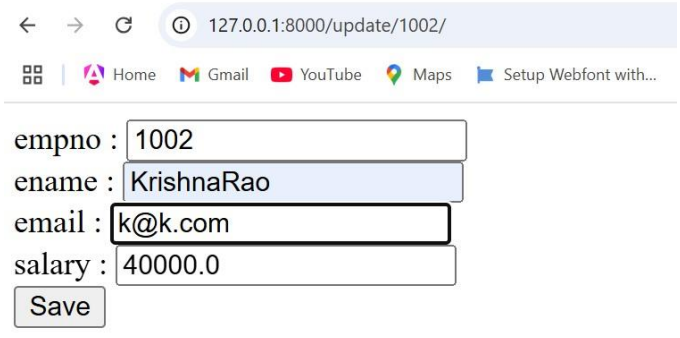
empno	empname	email	salary	action	
1001	Rama	rama@rama.cocm	50000.0	Edit	Delete
1002	Krishna	rama@rama.cocm	40000.0	Edit	Delete
1003	Siva	None	None	Edit	Delete
2222	King	abc@abc.com	123456.0	Edit	Delete
1001	Smith	smith@gmail.com	25000.0	Edit	Delete

Note: When user clicks on Delete button then that particular row will be deleted The following screen shows the latest data when user clicks on Delete button which has empno 1001



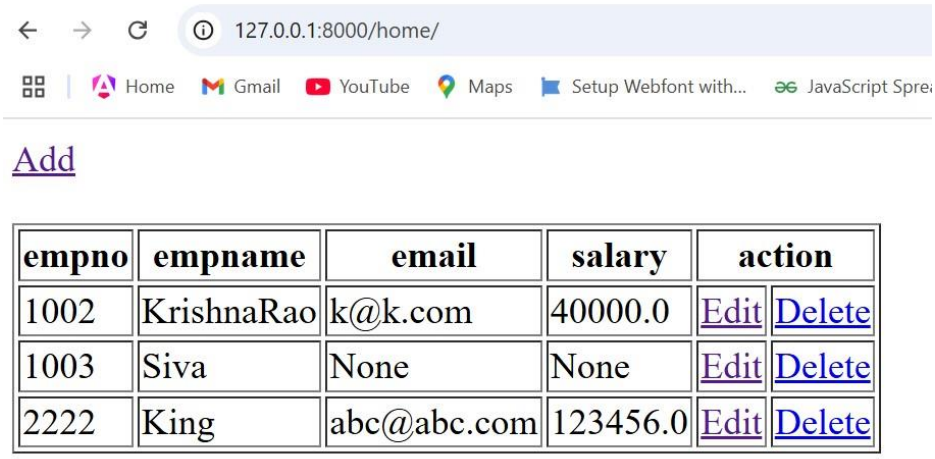
empno	empname	email	salary	action	
1002	Krishna	rama@rama.cocm	40000.0	Edit	Delete
1003	Siva	None	None	Edit	Delete
2222	King	abc@abc.com	123456.0	Edit	Delete

When user clicks on Edit button it will redirect to update page and allows the user to make necessary changes in the data. The following screen appears when user clicks on the row which has empno 1002



-

After making changes click on Save button to redirect to home page with updated data as follows



-